SUTRADHARA

Focus Agent — Complete Implementation Prompt

A specialist-grade engineering specification for adding intelligent, priority-aware focus scheduling to your multi-agent productivity orchestrator.

Framework	Pattern	Estimated effort
Google ADK + FastAPI	4th specialist under Manager	5–7 days focused build

1. Context & Problem Being Solved

Sutradhara currently operates as a reactive system — it executes commands when asked. The Focus Agent transforms it into a proactive system that reasons about your entire life simultaneously and protects what matters most.

The specific pain point this feature solves: you have multiple high-stakes parallel commitments — hackathons, learning tracks with hard deadlines, self-directed projects — and no system that synthesizes across all of them to tell you what to work on and when. Existing calendar tools hold your events. Notion holds your tasks. Neither does the math. Neither notices that your June 24 Udacity deadline is at risk because three hackathons in April and May will consume 15–20 days of potential study time.

> *The Focus Agent is the first component in Sutradhara that reads your full context — calendar, tasks, priorities, deadlines — simultaneously and produces an optimized weekly schedule before touching anything. It plans first, confirms with you, then acts.*

1.1 What the Focus Agent Does

Capability	Description
Priority ingestion	Accepts your stated priority weights (e.g. GenAI hackathon > Udacity Agent Builder > Linux Kernel) and stores them in a new UserPriorities table in SQLite.
Deadline math	Calculates hours remaining vs hours available for each commitment, accounting for calendar events that will consume focus time.
Conflict-aware scheduling	Reads existing Google Calendar events (hackathons, meetings) before proposing any focus blocks, never double-booking.
Optimized block generation	Proposes specific focus sessions: what to work on, when, for how long, in what order.
Plan-confirm-execute flow	Always shows the proposed plan in plain English before creating any calendar events. You approve, then it acts.
Weekly recalibration	Every Monday, reads what actually completed vs what was planned, adjusts pace estimates, re-proposes the week.
Progress tracking	Updates task/goal progress in SQLite after each session so pace estimates improve over time.

2. Architecture — Where It Plugs In

The Focus Agent slots in as a fourth specialist under your existing Manager Agent. No changes are required to the Manager, Calendar Specialist, Notion Specialist, or Planner. It reuses the existing MCP connections to Google Calendar and Notion via the Calendar Specialist and Notion Specialist respectively — it does not connect to external APIs directly.

2.1 Agent Hierarchy After Addition

Agent	Role
Manager (root)	Routes: if query is about focus/priorities/scheduling my week → delegates to Focus Agent. All other routing unchanged.
Focus Agent (new)	Reads priorities from SQLite + calendar from Calendar Specialist + tasks from Notion Specialist. Reasons. Produces a plan. On confirmation, calls Calendar Specialist to block time.
Calendar Specialist	Unchanged. Focus Agent calls it to (a) read existing events before planning, (b) create focus blocks after approval.
Notion Specialist	Unchanged. Focus Agent calls it to read task lists and update progress after sessions complete.
Planner Agent	Unchanged. Handles cross-tool multi-step workflows unrelated to focus scheduling.

2.2 New SQLite Tables Required

Add three tables to your existing SQLAlchemy models in app/models.py (or equivalent):

UserPriorities

Column	Type + Purpose
id	Integer primary key
goal_name	String — e.g. 'GenAI APAC Challenge', 'Udacity Agent Builder'
priority_rank	Integer — 1 = highest priority. User-defined ordering.
weekly_hours_target	Float — desired hours per week for this goal
total_hours_remaining	Float — estimated total work left (e.g. 20 for Udacity)
deadline	DateTime (nullable) — hard deadline if applicable
notes	String — free text context the agent uses when reasoning
is_active	Boolean — soft delete / pause without removing
created_at / updated_at	DateTime — for audit and tracking

FocusBlocks

Column	Type + Purpose
id	Integer primary key
goal_id	FK to UserPriorities
calendar_event_id	String — Google Calendar event ID (for undo support)
scheduled_start	DateTime
scheduled_end	DateTime
duration_minutes	Integer
status	Enum: proposed | confirmed | completed | skipped | rescheduled
actual_start / actual_end	DateTime (nullable) — filled in when marked complete
notes	String — what was planned to be done in this block
created_at	DateTime

ProgressLog

Column	Type + Purpose
id	Integer primary key
focus_block_id	FK to FocusBlocks
goal_id	FK to UserPriorities
hours_completed	Float — actual time worked
completion_notes	String — what was accomplished
pace_adjustment	Float — ratio of actual vs estimated (used to calibrate future estimates)
logged_at	DateTime

3. Focus Agent System Prompt

This is the complete system prompt to pass to the Focus Agent when initializing it in Google ADK. Copy this verbatim into your agent definition. Comments in [brackets] are implementation notes for you, not part of the prompt.

FOCUS AGENT SYSTEM PROMPT — COPY VERBATIM INTO YOUR ADK AGENT DEFINITION

> *Paste the following as the `instruction` parameter when defining the FocusAgent in Google ADK. Replace placeholder references with your actual tool names.*

You are the Focus Agent for Sutradhara, a multi-agent productivity orchestrator. Your sole purpose is to help the user protect their most important work by reasoning across their full context — calendar, tasks, priorities, and deadlines — and producing an optimized, realistic weekly focus schedule.

IDENTITY AND ROLE

You are a thinking partner, not a command executor. You never create calendar events without explicit user approval. You reason out loud, show your work, and explain every scheduling decision so the user understands and trusts the plan.

Your personality: direct, honest, slightly urgent when deadlines are at risk. You do not sugarcoat. If the user is behind on a commitment, you say so clearly with the math. You respect the user's autonomy — you propose, they decide.

TOOLS AVAILABLE TO YOU

- read_user_priorities(): Read all active goals from UserPriorities table sorted by priority_rank
- read_focus_blocks(date_from, date_to): Read scheduled and completed focus blocks in a date range
- read_progress_log(goal_id): Read completion history and pace data for a specific goal
- set_user_priority(goal_name, priority_rank, weekly_hours_target, total_hours_remaining, deadline, notes): Create or update a priority entry
- list_calendar_events(start_date, end_date): Call Calendar Specialist to read all existing events in a window
- find_free_slots(date, min_duration_minutes): Call Calendar Specialist to find available focus windows on a given day
- create_focus_block(goal_id, start_datetime, end_datetime, notes): Create a FocusBlock record (status=proposed) and return it for user review
- confirm_focus_block(block_id): Change status to confirmed and call Calendar Specialist to create the Google Calendar event. Returns calendar_event_id.

- mark_block_complete(block_id, hours_completed, completion_notes): Log progress, update hours_remaining on goal, update pace estimate
- read_notion_tasks(filter): Call Notion Specialist to read tasks filtered by project or status
- generate_weekly_plan(): Execute the full planning cycle: read priorities, read calendar, compute deadline math, find free slots, propose focus blocks. Returns a structured plan for user review.

REASONING PROCESS — FOLLOW THIS EXACTLY WHEN PLANNING

When asked to plan the user's week or schedule focus time, follow this sequence every time without skipping steps:

1. Step 1 — Load context. Call read_user_priorities() to get all active goals with their deadlines and hour estimates. Call list_calendar_events(today, today+14days) to understand what time is already committed.
2. Step 2 — Do the deadline math. For each goal with a deadline, compute: weeks_remaining = (deadline - today) / 7. net_available_weeks = weeks_remaining minus the number of weeks containing multi-day hackathons or travel. required_hours_per_week = total_hours_remaining / net_available_weeks. Flag any goal where required_hours_per_week > weekly_hours_target as AT RISK.
3. Step 3 — Establish priority order. Use priority_rank from UserPriorities. The user's stated priorities always override your judgment about what seems most important. If no priorities are set, ask before planning.
4. Step 4 — Find available slots. For each day in the planning window (next 7 days by default), call find_free_slots() to identify blocks of 90+ minutes not occupied by existing events. Prefer morning slots (before noon) when available. Never schedule focus blocks during existing events, meal times (12:00–13:00), or late evening (after 21:00) unless the user has specifically requested this.
5. Step 5 — Allocate time to goals. Starting with priority_rank 1, fill available slots. Allocate the weekly_hours_target for each goal before moving to the next priority. If there is not enough time to meet all weekly targets, allocate in priority order and explicitly tell the user which goals will be under-served this week and by how much.
6. Step 6 — Generate the plan. Produce a structured plan showing: (a) a risk summary for each goal, (b) a day-by-day schedule with specific focus blocks labeled by goal, (c) the total hours allocated per goal this week, (d) any goals that could not be fully accommodated and why.
7. Step 7 — Present and confirm. Show the plan in plain English. Do not create any calendar events yet. Ask: 'Does this look right? I will block these times in your calendar once you confirm.'
8. Step 8 — Execute only after confirmation. Call confirm_focus_block() for each approved block. Report the calendar event IDs created so the user can verify.

LANGUAGE AND OUTPUT FORMAT

When presenting a plan, always use this structure:

> *RISK SUMMARY[Goal name] — [X hours remaining, Y weeks to deadline, Z hrs/week needed] — STATUS: ON TRACK / AT RISK / CRITICALWEEK PLANMonday: [Goal name] — 9:00am–11:00am (2hrs) — [What to work on]Tuesday: [Goal name] — 9:00am–10:30am (1.5hrs) .....TOTAL ALLOCATION[Goal 1]: Xhrs (target: Yhrs) — [delta][Goal 2]: Xhrs (target: Yhrs) — [delta]UNMET TARGETS[Goal N] is short by X hours this week because [reason]. I recommend [mitigation].*

IMPORTANT CONSTRAINTS — NEVER VIOLATE THESE

- Never create calendar events without the user saying 'confirm', 'yes', 'do it', 'go ahead', or equivalent explicit approval.
- Never tell the user everything is fine if deadline math shows a goal is at risk. Be honest.
- Never schedule more than 4 hours of deep work in a single day without asking first.
- Never assume which hours are available — always call find_free_slots() before proposing times.
- Never overwrite or move existing calendar events. Focus blocks fill gaps only.
- Always support undo: every focus block created via confirm_focus_block() must store the Google Calendar event ID in FocusBlocks.calendar_event_id so it can be reversed via the existing Rollback Service.
- If the user says 'I want to focus more on X this week', update the priority_rank and weekly_hours_target for that goal immediately, then re-run the planning cycle from Step 1.
- If the user says 'skip [goal] this week', set that goal's weekly_hours_target to 0 for the current planning cycle only — do not permanently modify the UserPriorities record.

PROACTIVE BEHAVIORS

The APScheduler already runs proactive tasks in Sutradhara. Register the following scheduled jobs for the Focus Agent:

- Every Monday 7:00am: Run generate_weekly_plan() automatically and send the output as a briefing. Do not create any blocks until the user confirms.
- Every Friday 6:00pm: Run a weekly review — compare planned vs actual focus blocks, calculate hours completed per goal, update pace estimates, flag any goals that fell behind and need catch-up next week.
- 48 hours before any deadline: Alert the user with exact hours remaining, exact hours needed per day to finish, and a proposed intensive schedule if the goal is at risk.

END OF SYSTEM PROMPT

4. Manager Agent — Routing Update

Add the following routing instructions to the Manager Agent's existing system prompt. Insert after the existing specialist descriptions:

> *Add this block to your Manager Agent's instruction string, after the description of the Planner Agent:*

FOCUS AGENT — route to this specialist when the user's query is about any of the following:

- Setting or updating priorities ('I want to focus on X', 'make Y my top priority this week')
- Scheduling focus time, deep work blocks, or study sessions
- Requesting a weekly plan or schedule
- Asking what they should work on or in what order
- Reporting progress on a goal ('I finished 2 hours of Udacity today')
- Asking about deadline risk or whether they are on track
- Requesting a weekly review or progress summary
- Adjusting existing focus blocks ('move my Linux session to Thursday')

Do NOT route to the Focus Agent for: general calendar queries (use Calendar Specialist), general task management (use Notion Specialist), or multi-tool workflows not related to focus scheduling (use Planner Agent).

5. Tools Implementation Guide

Each tool the Focus Agent uses must be implemented as either a direct SQLite function or a delegation call to an existing specialist. Here is the implementation guidance for each:

5.1 Tools That Read/Write SQLite Directly

Implement these as Python functions decorated with @tool in Google ADK, accessing your SQLAlchemy session:

read_user_priorities()

```
SELECT * FROM user_priorities WHERE is_active = TRUE ORDER BY priority_rank ASC
```

Returns a list of dicts. Include all columns. The agent uses deadline, total_hours_remaining, and weekly_hours_target for its math.

set_user_priority(...)

```
UPSERT into user_priorities ON CONFLICT(goal_name) DO UPDATE SET ...
```

Use SQLAlchemy merge() or equivalent. Always update updated_at. Validate that priority_rank is unique — if a duplicate rank is given, shift existing records down.

create_focus_block(goal_id, start, end, notes)

Insert into focus_blocks with status='proposed'. Return the full record including id. Do not touch Google Calendar at this stage.

confirm_focus_block(block_id)

1. Fetch the FocusBlock record. 2. Call Calendar Specialist's create_event() with the block's start/end/title (title = goal_name from UserPriorities). 3. Store the returned calendar_event_id in focus_blocks.calendar_event_id. 4. Update status to 'confirmed'. 5. Log to ActionLog (existing table) as a reversible action so the Rollback Service can undo it.

mark_block_complete(block_id, hours_completed, completion_notes)

1. Update FocusBlock status to 'completed', set actual_end = now(). 2. Insert into ProgressLog. 3. Calculate pace_adjustment = hours_completed / planned_duration_hours. 4. Update UserPriorities.total_hours_remaining -= hours_completed. 5. Update pace estimate for future planning.

5.2 Tools That Delegate to Existing Specialists

These are thin wrappers that call your existing agent tools. Implement as @tool functions that invoke the sub-agent:

Focus Agent Tool	Delegates To
list_calendar_events(start, end)	Calendar Specialist — list_events(start, end)
find_free_slots(date, min_duration)	Calendar Specialist — find_free_slots(date) then filter by min_duration
read_notion_tasks(filter)	Notion Specialist — query_notion_database(filter)

6. User-Facing Commands to Support

Design the Focus Agent to understand all of the following natural language inputs. These are the commands you should test before submitting or shipping:

6.1 Priority Setup

- 'My priorities right now are: 1. GenAI hackathon, 2. Udacity Agent Builder (deadline June 24, 20hrs left), 3. Udacity Agent Programmer (deadline June 24, 20hrs left), 4. Linux Kernel roadmap'
- 'Make the Udacity tracks my top focus for the next 3 weeks'
- 'Pause the Linux Kernel roadmap for now, I'll come back to it'
- 'I want to put 10 hours per week into Udacity from now until June 24'

6.2 Planning

- 'Plan my week'
- 'Schedule focus time for this week'
- 'What should I work on today?'
- 'Am I on track to finish Udacity by June 24?'
- 'Show me the deadline math for all my goals'

6.3 Execution and Adjustment

- 'Yes, confirm the plan' / 'Go ahead' / 'Block those times'
- 'Move my Linux session to Friday afternoon instead'
- 'Skip Udacity this week, I'm heads-down on the hackathon'
- 'I finished 3 hours of the Agent Builder course today'

6.4 Review

- 'How did I do this week?'
- 'Show me my focus summary for the last 2 weeks'
- 'How many hours do I still need to complete Udacity?'

7. Recommended Build Order

Build in this exact sequence. Each step is independently testable before proceeding to the next.

Day	What to Build
Day 1	Database: Add UserPriorities, FocusBlocks, ProgressLog tables to models.py. Run Alembic migration. Verify tables exist in SQLite.
Day 2	Tools: Implement read_user_priorities(), set_user_priority(), create_focus_block() as standalone Python functions. Test each with direct SQLite queries.
Day 3	Agent definition: Create FocusAgent in Google ADK with the system prompt from Section 3. Register read_user_priorities and set_user_priority tools only. Test: 'My top priority is GenAI hackathon' — verify it writes to DB.
Day 4	Planning logic: Add list_calendar_events and find_free_slots delegation tools. Test generate_weekly_plan() with your real calendar data. Verify the risk math is correct against your actual Udacity deadlines.
Day 5	Execution: Add confirm_focus_block() with Calendar Specialist delegation and ActionLog integration. Test the full plan-confirm-execute flow. Verify undo works via Rollback Service.
Day 6	Progress tracking: Add mark_block_complete() and ProgressLog writes. Add weekly review logic. Test the Monday briefing and Friday review scheduled jobs.
Day 7	Manager routing: Add Focus Agent routing rules to Manager prompt. Test that 'plan my week' routes correctly. Test all commands from Section 6. Fix edge cases.

8. Testing Checklist

Before considering the Focus Agent complete, verify every item in this list:

8.1 Database

- UserPriorities table created and accepting inserts
- FocusBlocks table created with FK to UserPriorities
- ProgressLog table created with FK to FocusBlocks
- Alembic migration runs without error on a fresh DB

8.2 Core Planning Logic

- Deadline math is correct: given 20hrs remaining and 12 weeks, with 2 hackathon weeks, it correctly calculates 20/10 = 2hrs/week required
- AT RISK flag triggers when required_hours_per_week > weekly_hours_target
- Free slot detection never proposes times that overlap existing calendar events
- Priority ordering is respected: goal rank 1 gets time allocated before rank 2

8.3 Plan-Confirm-Execute Safety

- Agent NEVER creates calendar events without explicit confirmation
- confirm_focus_block() stores calendar_event_id in FocusBlocks table
- Undo via Rollback Service successfully deletes the Google Calendar event
- FocusBlock status transitions correctly: proposed → confirmed → completed

8.4 Manager Routing

- 'Plan my week' routes to Focus Agent, not Planner
- 'What meetings do I have tomorrow' routes to Calendar Specialist, not Focus Agent
- 'Create a Notion task' routes to Notion Specialist, not Focus Agent
- 'I want to focus on Udacity this week' routes to Focus Agent

8.5 Proactive Features

- Monday 7am job triggers and produces a plan (check APScheduler registration)
- Friday 6pm job triggers and produces a review
- 48-hour deadline alert fires correctly (test with a mock deadline 48 hours from now)

9. Future Vision — Where to Take This Next

Once the core Focus Agent is working, these are the highest-leverage extensions — in order of impact:

Feature	Why It Matters
Pace learning	Track actual vs estimated hours per goal over 4+ weeks. Automatically adjust weekly_hours_target based on your real completion rate, not your optimistic estimate.
Energy-aware scheduling	Let the user declare 'I do my best deep work before noon'. Respect this in slot selection. Over time, correlate time-of-day with completion rate from ProgressLog.
Gmail integration	Extract commitments from emails ('I'll send that by Friday') and auto-create UserPriority entries or Notion tasks. This closes the loop on promises made outside your system.
Focus mode protection	When a confirmed focus block is active, the Manager Agent declines meeting requests that conflict with it and proposes alternatives automatically.
Team coordination	When two users both run Sutradhara, their Focus Agents negotiate meeting times that respect each other's focus blocks before proposing to the humans.
Voice interface	The Focus Agent is the ideal entry point for voice — 'Hey, what should I work on right now?' is the most natural productivity query in the world.
IST / timezone awareness	Explicitly store the user's timezone in UserPriorities or a UserProfile table. All scheduling math should use local time, especially important for users in India where morning slots before 9am may conflict with global team calls.

Built for yourself first. Built for the world second.

The best productivity tools are ones their creators actually use.